



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N.º 10

NOMBRE COMPLETO: Rosas Cañada Abraham

N.º de Cuenta: 318307866

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 10/05/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ejercicio 1: Animar las alas de la nave para que mientras que recorre el ciclo del ejercicio vayan aleteando adecuadamente, compartir capturas de pantalla donde se vea la impresion de los keyframes guardados o reproducidos en consola o guardados en un archivo de tipo txt y compartir los keyframes para que yo pueda reproducir su animacion.

Este ejercicio parte del codigo de keyframes desarrollado en sesiones anteriores y le anade dos animaciones propias del modelo de la nave que se ejecutan automaticamente durante todo el ciclo, sin depender de los cuadros clave: el aleteo continuo de las alas y la rotacion de las helices. La animacion por keyframes sigue siendo la responsable de la trayectoria global (recorrido senoidal de ida, giro de 180 grados en el extremo, regreso por el mismo camino y giro final de cierre), mientras que las alas y las helices se mueven independientemente en cada frame mediante funciones matematicas. Adicionalmente se modifiko la funcion saveFrame para que cada vez que el usuario presione la tecla L se imprima el estado del cuadro tanto en consola como en un archivo de texto plano llamado keyframes.txt, lo que permite compartir la secuencia capturada para que pueda reproducirla un tercero.

Cambios principales en main.cpp:

Para que las alas tengan un movimiento continuo y suave durante todo el recorrido se introdujo una funcion auxiliar llamada OscilarEntre que devuelve un valor que sube y baja sin cortes usando la funcion seno. La salida se mapea a un rango definido por dos angulos limite, uno trasero negativo (-5 grados) y uno frontal positivo (60 grados), lo que produce el efecto de aleteo natural. Esta funcion se invoca cada frame con el tiempo acumulado en naveOscAla y su resultado se asigna a la variable naveAngAla, que despues se aplica como rotacion en el eje Y al ala izquierda con signo positivo y al ala derecha con signo negativo, lo que produce un aleteo simetrico en el que ambas alas se abren y se cierran al mismo tiempo respecto al eje central de la nave.

Funcion auxiliar OscilarEntre y variables de animacion:

```
// Animacion propia de la nave: helices y alas
float naveVelHelice = 7800.0f;
float naveVelAla = 20.5f;
float naveAlaFrente = 60.0f;
float naveAlaAtras = -5.0f;
float naveRotHelice = 0.0f;
float naveOscAla = 0.0f;
float naveAngAla = 0.0f;

// Devuelve un valor que oscila suavemente entre minimo y maximo
// usando una funcion seno parametrizada por el tiempo acumulado.
float OscilarEntre(float minimo, float maximo, float tiempo)
{
    float t = (sin(tiempo) + 1.0f) * 0.5f;
    return minimo + (maximo - minimo) * t;
}
```

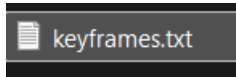


Figura 1. Captura del archivo keyframes.txt generado al guardar todos los cuadros con la tecla L y de la consola mostrando los mensajes Frame N guardado durante el proceso de captura por teclado.

Actualizacion del aleteo y la rotacion dentro del loop principal:

Las dos animaciones propias de la nave se actualizan al inicio del while antes del bloque de teclas y antes de la llamada a animate(), para que en cada iteracion del render la posicion de las alas y de las helices ya este lista cuando se construya la matriz del modelo. La rotacion de las helices acumula 7800 grados por segundo en la variable naveRotHelice y se reinicia al pasar de 360 para evitar que la variable crezca indefinidamente; el aleteo acumula el tiempo transcurrido en naveOscAla y obtiene el angulo actual mediante OscilarEntre.

```
// Actualiza la rotacion continua de las helices y el aleteo de las alas.
// Las helices giran a una velocidad alta y constante; las alas oscilan
// suavemente entre el angulo trasero y el frontal con una funcion seno.
naveRotHelice += naveVelHelice * deltaTime;
if (naveRotHelice >= 360.0f)
    naveRotHelice -= 360.0f;

naveOscAla += naveVelAla * deltaTime;
naveAngAla = OscilarEntre(naveAlaAtras, naveAlaFrente, naveOscAla);
```

Render de las alas con jerarquia y aleteo opuesto:

El bloque de render de la nave aplica primero la traslacion y el giro globales que controlan los keyframes y guarda la matriz resultante en modelaux. A partir de esa matriz se renderiza cada pieza con su offset local: el cuerpo de la nave se dibuja primero, luego el ala izquierda con una rotacion en Y de +naveAngAla, y a continuacion el ala derecha con una rotacion de -naveAngAla, lo que produce el aleteo simetrico. Las helices se posicionan con sus offsets propios (naveOffsetHeliceIzq y naveOffsetHeliceDer) y aplican naveRotHelice como rotacion continua sobre el eje X. Toda la jerarquia hereda la transformacion global de la nave, por lo que cuando los keyframes mueven la nave a lo largo del recorrido, las piezas siguen aleteando y girando sin perder su anclaje al cuerpo.

```
// Ala izquierda con aleteo
model = modelaux;
model = glm::translate(model, naveOffsetAlaIzq);
model = glm::rotate(model, (naveRotAlaIzq.y + naveAngAla) * toRadians,
    glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(naveEscala, naveEscala, naveEscala));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ala_M.RenderModel();

// Ala derecha con aleteo opuesto al de la izquierda
model = modelaux;
model = glm::translate(model, naveOffsetAlaDer);
model = glm::rotate(model, (naveRotAlaDer.y - naveAngAla) * toRadians,
    glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(naveEscala, naveEscala, naveEscala));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Ala2_M.RenderModel();
```

Guardado de los keyframes en consola y en archivo de texto:

La funcion saveFrame se modifico para que ademas de imprimir el estado del cuadro en consola con printf, abra el archivo keyframes.txt en modo append (std::ios::app) y le agregue una linea con el indice del frame y los valores de x, y y giro. Para garantizar que cada ejecucion empiece con un archivo limpio, antes del while principal se abre el archivo en modo std::ios::trunc, lo que borra cualquier contenido previo. De esta forma, durante toda la sesion de captura por teclado, el archivo se va llenando automaticamente y al final puede compartirse para reproducir la animacion sin tener que capturar los frames de nuevo.

```
void saveFrame(void)
{
    if (FrameIndex >= MAX_FRAMES)
    {
```

```

        printf("No hay espacio para mas frames\n");
        return;
    }

    KeyFrame[FrameIndex].movAvion_x = movAvion_x;
    KeyFrame[FrameIndex].movAvion_y = movAvion_y;
    KeyFrame[FrameIndex].giroAvion = giroAvion;

    printf("Frame %d guardado: x=%.2f, y=%.2f, giro=%.2f\n",
        FrameIndex, movAvion_x, movAvion_y, giroAvion);

    // Guardar el frame tambien en un archivo de texto
    std::ofstream archivo("keyframes.txt", std::ios::app);
    if (archivo.is_open())
    {
        archivo << "Frame " << FrameIndex
            << " | x=" << movAvion_x
            << " | y=" << movAvion_y
            << " | giro=" << giroAvion << "\n";
        archivo.close();
    }

    FrameIndex++;
}

```



Figura 2. Reproduccion de la animacion completa con la nave recorriendo la trayectoria senoidal.

Limpieza del archivo keyframes.txt al iniciar el programa:

Antes de entrar al loop principal del while, justo despues de imprimir la lista de teclas en pantalla, se abre el archivo keyframes.txt en modo trunc y se cierra

inmediatamente. Esta operacion garantiza que el archivo este vacio al inicio de cada ejecucion del programa, evitando que se mezclen frames de sesiones anteriores con los nuevos.

```
// Borrar el archivo de keyframes anterior al iniciar
std::ofstream limpiar("keyframes.txt", std::ios::trunc);
limpiar.close();
```

Resumen de teclas para la ejecucion:

La siguiente tabla resume las teclas habilitadas durante la ejecucion del programa. Todas estan implementadas con deteccion de flanco (curr && !prev) para garantizar que cada pulsacion se registre una sola vez aunque la tecla se mantenga presionada, evitando saturar la consola con mensajes repetidos y los incrementos multiples por frame.

Tecla	Accion	Descripcion
ESPACIO	Reproducir / pausar	Inicia la reproduccion de la animacion con los keyframes capturados o la pausa si ya esta en curso.
L	Guardar frame	Captura el estado actual de la nave (x, y, giro) como un nuevo keyframe y lo escribe en consola y en keyframes.txt.
1	Mover X +1	Incrementa una unidad la posicion en X (avanza a la derecha).
2	Mover X -1	Decrementa una unidad la posicion en X (retrocede a la izquierda).
3	Mover Y +2	Sube dos unidades la posicion vertical.
4	Mover Y -2	Baja dos unidades la posicion vertical.
5	Girar +60	Acumula 60 grados de giro sobre el eje Y.
R	Reiniciar	Restablece la nave al estado del primer keyframe (frame 0) y detiene la reproduccion.
0	Habilitar replay	Permite reproducir nuevamente la animacion con la barra espaciadora.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar como

- Uno de los primeros problemas encontrados fue la orientación inicial de la nave. Aunque en el código se asignaba una rotación de 180 grados, visualmente el modelo no iniciaba apuntando en la dirección esperada. Esto se debía a que el modelo importado ya traía una orientación base desde su archivo OBJ, por lo que la rotación aplicada en OpenGL no coincidía directamente con la referencia visual deseada. La solución consistió en separar la rotación lógica de la animación de una corrección fija del modelo, de manera que la nave pudiera iniciar visualmente en la orientación correcta sin alterar la lógica interna de los keyframes.
- Otro problema importante fue el eje de rotación. En versiones previas, la nave se estaba girando sobre el eje Z, lo que provocaba un comportamiento incorrecto, ya que parecía inclinarse o rotar de manera distinta a la intención del ejercicio. Para corregirlo, se cambió la transformación para que el giro se realizara sobre el eje Y, que es el eje adecuado para representar cambios de dirección horizontales en la escena. Con esto, la nave pudo girar correctamente en los extremos del recorrido.
- También surgió un problema relacionado con el sentido del movimiento. Al reproducir la trayectoria, la nave daba la impresión de ir en reversa, aun cuando los desplazamientos en posición eran correctos. Esto se resolvió ajustando la orientación general del modelo y los ángulos de los keyframes, de forma que la dirección visual de la nave coincidiera con la trayectoria que estaba recorriendo. Este ajuste fue especialmente importante al inicio del ciclo, en el giro del extremo derecho y en el giro final al regresar al punto de origen.
- Un problema adicional apareció en la interpolación de la rotación. Cuando se intentaba volver de 360 grados a 180 grados, en algunos casos el comportamiento no reflejaba visualmente el giro esperado de 180 grados. Esto ocurre porque en animación por keyframes no basta con que dos orientaciones sean visualmente equivalentes; también es necesario que los valores angulares permitan una transición coherente durante la interpolación. Para resolverlo, se organizaron los cuadros clave con valores angulares acumulados, permitiendo que la nave realizara giros de 180 grados claramente definidos en cada cambio de dirección.
- Otro de los problemas principales fue que al finalizar la animación la nave no siempre regresaba exactamente a su estado inicial. En algunos intentos terminaba con una orientación distinta o con una pequeña variación en su transformación final. La solución consistió en agregar un cuadro clave final que restableciera explícitamente la orientación necesaria para dejar a la nave en la misma condición visual con la que comenzó el ciclo. Además, se reforzó la rutina de reinicio para que, al terminar la reproducción, el sistema volviera a cargar el primer keyframe.

- Durante la implementación también se detectó que la velocidad de la animación afectaba la percepción general de toda la escena. Al principio se estaba utilizando una lógica donde varios elementos parecían avanzar bajo un mismo ritmo global, lo cual provocaba la sensación de que incluso el movimiento del POV o de la cámara se volvía más lento. Esto era problemático porque la cámara no debía depender directamente de la velocidad de la nave ni de las texturas animadas. Para corregirlo, se separaron las velocidades en variables independientes, asignando un control propio para la cámara, otro para la animación de keyframes, otro para el coche, otro para las llantas y otros adicionales para las texturas animadas. De esta forma, cada sistema pudo ajustarse sin alterar a los demás.
- Otro detalle que generó problemas fue el manejo del teclado. En una implementación inicial, al mantener presionada una tecla se repetían múltiples acciones en consola y se alteraban valores varias veces dentro de un mismo intervalo de interacción. Esto complicaba tanto la reproducción como el guardado manual de keyframes. La solución fue aplicar detección por flanco de tecla, es decir, ejecutar la acción únicamente cuando la tecla pasa de no presionada a presionada. Gracias a este cambio, fue posible reproducir la animación, guardar cuadros y modificar variables de manera mucho más controlada.
- También hubo dificultades al definir la trayectoria senoidal de ida y vuelta. Aunque el boceto servía como referencia, trasladarlo a valores concretos de posición en X y Y para 23 cuadros clave requería mantener simetría entre la ida y el regreso, conservar el ritmo visual del movimiento y al mismo tiempo coordinar los giros de 180 grados en los extremos. La solución fue construir una secuencia ordenada de keyframes que describiera de manera explícita cada ascenso, descenso, punto medio, extremo derecho, retorno y cierre del ciclo.
- En el caso específico del ejercicio de las alas, uno de los problemas fue sincronizar el aleteo con el movimiento general de la nave. Si el aleteo dependía directamente del desplazamiento global, se percibía rígido o fuera de tiempo en algunas partes del recorrido. Para resolver esto, se decidió tratar la animación de las alas como un movimiento secundario controlado de forma independiente, de modo que pudieran seguir aleteando adecuadamente durante todo el ciclo sin depender por completo de la velocidad del desplazamiento principal.

3.- Conclusion:

Este ejercicio permitio combinar dos tipos de animacion en un mismo modelo: la animacion controlada por keyframes que define la trayectoria global de la nave en el escenario, y la animacion procedural propia del objeto que se ejecuta de manera independiente en cada frame mediante funciones matematicas como el seno. Esta separacion es importante porque demuestra que un modelo complejo no se anima como un bloque rigido sino como una jerarquia de piezas, donde cada pieza puede tener su propio comportamiento y todas heredan la transformacion del padre. La adiccion de la escritura a archivo de texto resulto particularmente util porque permite portar la animacion entre sesiones, compartirla con el profesor para verificacion y eventualmente leer los keyframes de vuelta al programa para reproducirlos sin tener que capturarlos cada vez. La leccion principal del ejercicio es que el sistema de keyframes y la animacion procedural no son alternativas mutuamente excluyentes, sino capas complementarias que juntas producen escenas mucho mas ricas que cualquiera de las dos por separado.

Bibliografia:

- Khronos Group. (s.f.). *Keyframe Animation*. OpenGL Wiki. Recuperado el 9 de mayo de 2026, de https://www.khronos.org/opengl/wiki/Keyframe_Animation
- Megabyte Softworks. (s.f.). *Animation Pt.1 - Keyframe MD2*. OpenGL 4 Tutorials. Recuperado el 9 de mayo de 2026, de <https://www.mbsoftworks.sk/tutorials/opengl4/030-animation-pt1-keyframe-md2/>
- de Vries, J. (s.f.). *Skeletal Animation*. LearnOpenGL. Recuperado el 9 de mayo de 2026, de <https://learnopengl.com/Guest-Articles/2020/Skeletal-Animation>
- GLM Authors. (s.f.). *OpenGL Mathematics (GLM) Manual: Transformations and Matrix Stack*. GLM Documentation. Recuperado el 9 de mayo de 2026, de <https://github.com/g-truc/glm/blob/master/manual.md>
- cppreference. (s.f.). *std::ofstream and std::ios_base::openmode (app, trunc, out)*. C++ Reference. Recuperado el 9 de mayo de 2026, de https://en.cppreference.com/w/cpp/io/basic_ofstream